Clase 03: Memoria y Caché

IIC3533 Computación de Alto Rendimiento

Ignacio Labarca – ignacio.labarca@uc.cl

Departamento de Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica de Chile

Jueves 12 de Marzo de 2026

Rendimiento y Memoria

Latencia y Ancho de Banda

Memoria Caché

Mapeo y Políticas de Caché

Caché y Multicore

cpu-bound vs memory-bound

La velocidad del procesador no siempre es el cuello de botella.

- Algoritmos *cpu-bound*: limitados por la velocidad de procesamiento.

  La memoria entrega datos a tiempo, pero el procesador no puede calcular más rápido.

- Algoritmos *memory-bound*: limitados por la velocidad de acceso a memoria.

  El procesador queda ocioso esperando datos.

¿Cómo distinguirlos? Mediante la **Intensidad Aritmética**.

Intensidad Aritmética

La **Intensidad Aritmética** (AI) mide cuánto cómputo se realiza por cada byte transferido:

$$\mathrm{AI} = \frac{\text{N° de operaciones (FLOP)}}{\text{Bytes transferidos desde memoria}}$$

- AI alta: muchos FLOP por byte. El programa es *cpu-bound*.
- AI baja: pocos FLOP por byte. El programa es *memory-bound*.

Ejemplo *cpu-bound*: GEMM

Multiplicación de matrices: $C \leftarrow A \cdot B + C$, con $A, B, C \in \mathbb{R}^{n \times n}$.

Cómputo

$2n^3$ operaciones

(cada elemento de C requiere n multiplicaciones y n sumas)

Datos

$3 \times 8n^2$ bytes

(3 matrices de $n \times n$ doubles)

$$\mathrm{AI} = \frac{2n^3}{24n^2} = \frac{n}{12} \quad \text{FLOP/Byte}$$

Con $n = 1000$: $\mathrm{AI} \approx 83$ FLOP/Byte. Es cpu-bound.[1]

[1] Asume que cada dato se carga una sola vez (matrices caben en caché) y que el procesador ejecuta las operaciones de punto flotante sin cuello de botella interno.

Ejemplo *memory-bound*: DAXPY

Operación vectorial: $y \leftarrow a \cdot x + y$, con $x, y \in \mathbb{R}^n$, a escalar.

Cómputo

$2n$ operaciones

(una multiplicación y una suma por elemento)

Datos

$3 \times 8n$ bytes

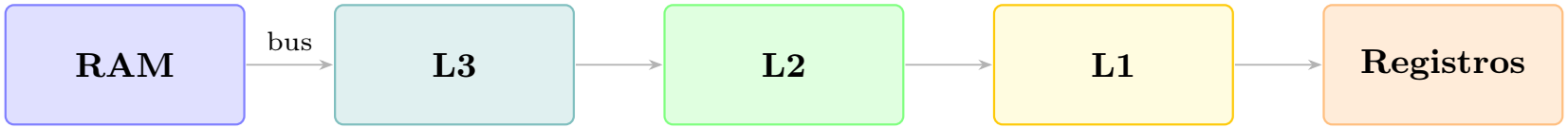
(leer x e y, escribir y)

$$\mathrm{AI} = \frac{2n}{24n} = \frac{1}{12} \approx 0{,}083 \quad \text{FLOP/Byte}$$

AI no depende de n y es muy baja. Es memory-bound.

Movimiento de datos: RAM → Procesador

El procesador no accede directamente a la RAM: los datos viajan por un **bus de memoria**.

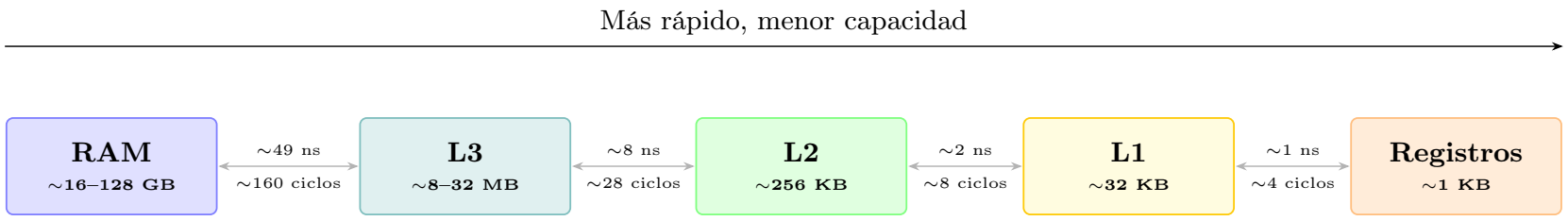


El rendimiento de este viaje se describe con dos métricas:

- **Latencia**: ¿cuánto tarda en llegar el primer byte?
- **Ancho de banda**: ¿a qué tasa llegan los bytes siguientes?

Jerarquía de memoria

Cada nivel es más rápido pero más pequeño:



El objetivo es maximizar la reutilización de datos en los niveles cercanos al procesador.

Memoria Caché

Entre la RAM y los registros existen varios niveles de **memoria caché**:

- Los movimientos caché-memoria se hacen por hardware (no por el programador).
- Se caracterizan por baja latencia y alto ancho de banda.
- Los datos se mantienen de manera temporal: la caché es pequeña.
- Existe L1d para datos y L1i para instrucciones. L1 y L2 suelen estar en el chip.

Latencia

Latencia (α): tiempo desde que se solicita un dato hasta que llega el primero.

- Se mide en ciclos o en nanosegundos (*ns*).
- Ejemplo típico (procesador a 3,5 GHz, 1 ciclo $\approx$ 0,3 *ns*):

Nivel	Latencia (ciclos)	Latencia (*ns*)
L1 cache	~ 4	~ 1
L2 cache	~ 12	~ 3
L3 cache	~ 40	~ 11
RAM	~ 200	~ 60

Durante la latencia, el procesador queda bloqueado (*memory stall*) a menos que pueda ejecutar otras instrucciones (*latency hiding*).

Latency Hiding

Técnicas para que el procesador no quede ocioso durante la latencia:

- ***Out-of-Order Execution***: el procesador ejecuta instrucciones independientes mientras espera datos.
- ***Prefetching***: el hardware (o compilador) anticipa accesos futuros y solicita los datos por adelantado.
- ***Multithreading* (SMT / HyperThreading)**[2]: cuando un hilo espera por memoria, otro hilo usa las unidades de ejecución.

Idealmente: *latencia oculta* $\approx$ trabajo útil durante la espera.

[2] **SMT**: **Simultaneous MultiThreading**.

Ancho de banda

Ancho de banda (BW): tasa a la que llegan datos *después* de la latencia inicial.

- Se mide en GB/s o en Byte/ciclo.
- Depende del **bus de memoria**:
  - Velocidad del bus (*bus speed*).
  - Ancho del bus (bits por transferencia).
  - Número de canales (dual-channel, quad-channel).

El tiempo total de transferencia de n bytes es:

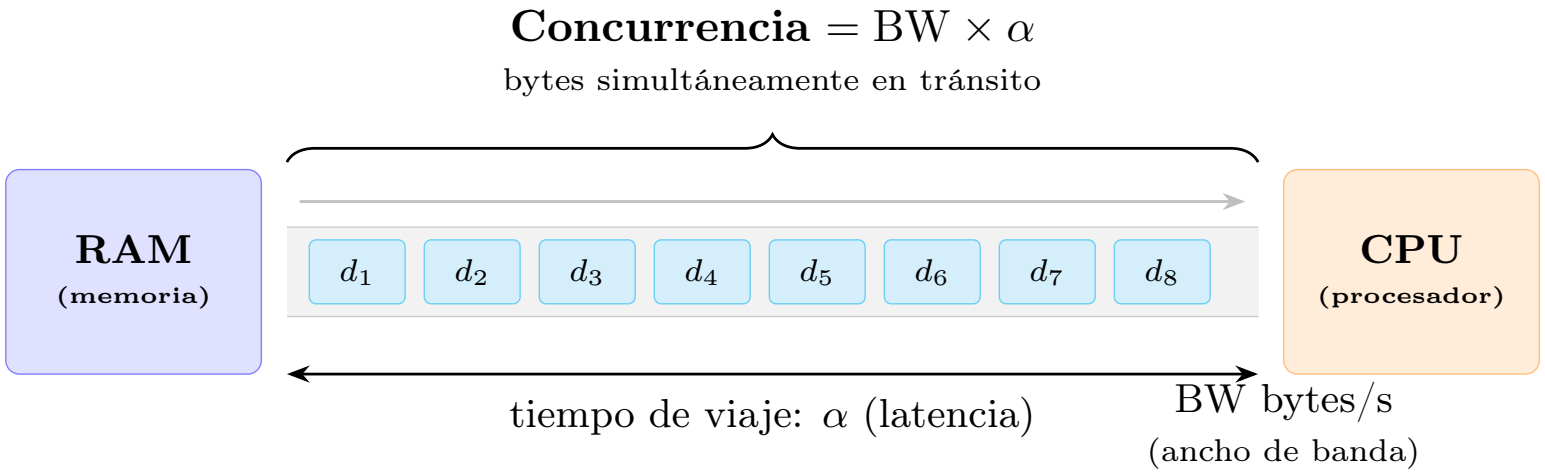
$$T(n) = \alpha + \beta\, n$$

α : latencia (tiempo de inicio)

$\beta = 1/\mathrm{BW}$: inverso del ancho de banda (tiempo por byte)

Concurrencia

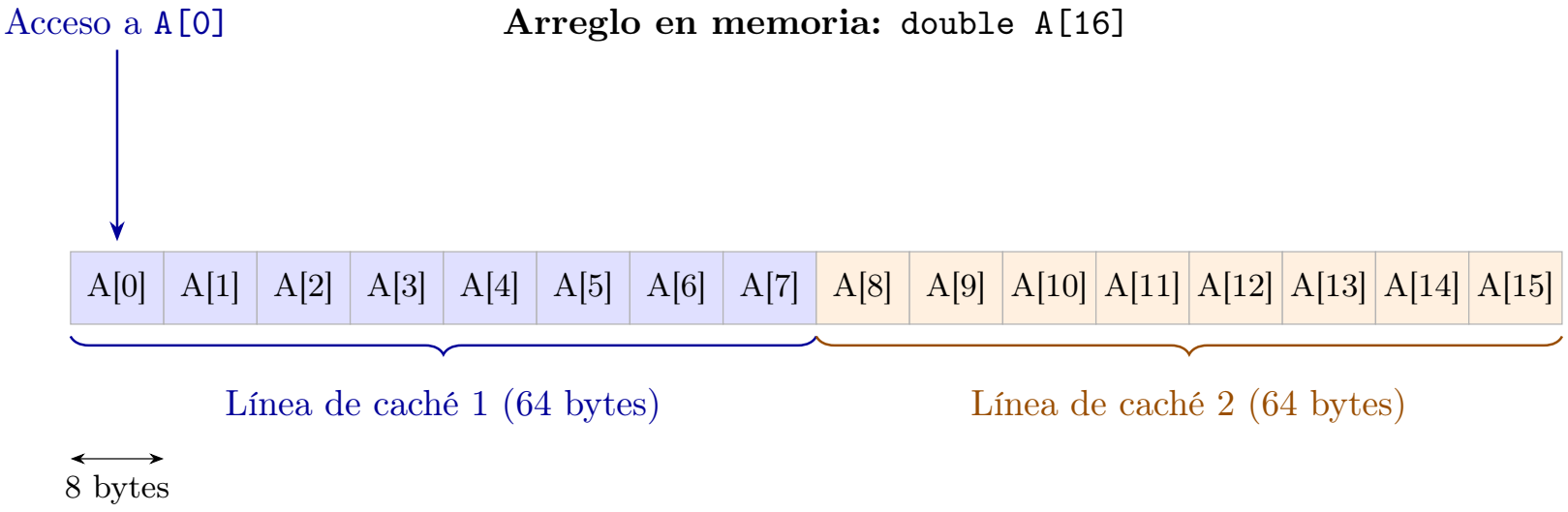
El máximo nivel de concurrencia se obtiene cuando todo el ancho de banda está en uso:



Si $\mathrm{BW} = 50$ GB/s y $\alpha = 60$ ns, entonces hay $\mathrm{BW} \times \alpha = 3000$ bytes en tránsito simultáneamente.

Línea de caché (*cacheline*)

Los datos no se mueven byte a byte, sino en bloques llamados **líneas de caché**:



- Al acceder a `A[0]`, el hardware carga la **línea completa**: `A[0]` a `A[7]`.
- Los accesos siguientes a `A[1]`, `A[2]`, ..., `A[7]` son inmediatos (ya están en caché).

Caché: reutilización de datos

La ventaja de tener memoria caché surge con reutilizar datos.

- Datos que ya están en memoria caché se acceden más rápido.
- Memoria caché es menor, los datos no se pueden quedar para siempre.
- Datos se copian en cada nivel en su viaje a los registros.

Caché: hit / miss

- Diremos *cache hit* cuando solicitemos datos que ya se encuentran en caché.

  Ejemplo: Solicitamos *x[0]*; la línea de caché trae 8 `double`s contiguos. Luego, *x[1]* ya está en L1: *hit*.

Caché: hit / miss

- Diremos *cache hit* cuando solicitemos datos que ya se encuentran en caché.

  Ejemplo: Solicitamos *x[0]*; la línea de caché trae 8 `double`s contiguos. Luego, *x[1]* ya está en L1: *hit*.

- Diremos *cache miss* cuando no podemos encontrar el dato en caché.

  Ejemplo: Solicitamos *x[0]*; la línea trae *x[0]* a *x[7]*. Luego, *x[100]* **no** está en esa línea: *miss*.

Caché miss

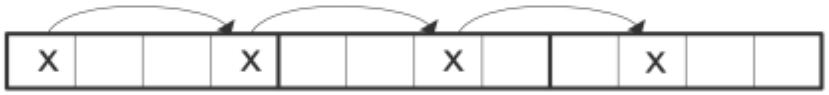
Existen distintos casos:

- **Compulsory cache miss**.
  Dato faltante.
- **Capacity cache miss**.
  Caché está lleno y hay que sobreescribir.
- **Conflict miss**.
  Ambos datos son necesarios y se quieren almacenar en el mismo lugar.
- En multicore: **invalidation miss**.
  El valor ha sido actualizado por otro núcleo.

Patrones de acceso y *stride*

No todos los accesos a memoria aprovechan la línea de caché igual:

- **Stride 1** (acceso contiguo): cada línea cargada se usa completa. Es eficiente.
- **Stride grande** (saltos): solo se usa un elemento por línea. Es desperdicio.



Precaución con los saltos en un for-loop: un stride de 8 `doubles` (64 bytes) significa que cada acceso carga una línea de caché nueva sin reutilizar la anterior.

Localidad espacial vs temporal

Los programas eficientes explotan dos tipos de **localidad**:

- **Localidad temporal**: un dato accedido recientemente se accederá de nuevo pronto.
  Ejemplo: variable de un bucle, acumulador.

- **Localidad espacial**: si se accede a una dirección, pronto se accederán direcciones cercanas.
  Ejemplo: recorrido secuencial de un arreglo.

La caché aprovecha ambas: trae una **línea completa** (espacial) y la mantiene **mientras sea útil** (temporal).

Ejemplo: orden de loops e impacto en caché

En C/C++, las matrices se almacenan por **filas** (*row-major*):

```
const int N = 4096;
std::vector<double> A(N * N); // memoria contigua

// Loop i,j: j varia rapido -> acceso contiguo -> cache hits
for (int i = 0; i < N; i++)
  for (int j = 0; j < N; j++)
    sum += A[i*N + j];

// Loop j,i: i varia rapido -> saltos de N*8 bytes -> cache misses
for (int j = 0; j < N; j++)
  for (int i = 0; i < N; i++)
    sum += A[i*N + j];
```

- Con $N = 4096$ y `double` (8B), cada fila ocupa 32 KiB.
- El loop j,i salta una fila completa entre accesos.
  Produce muchos cache misses.

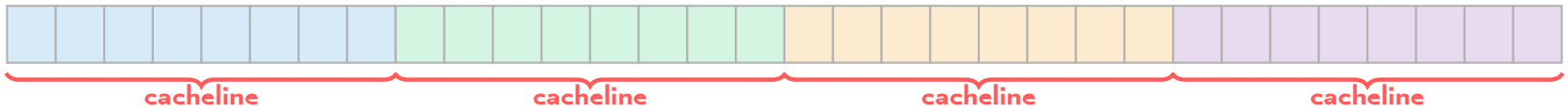
Loop i,j: acceso contiguo

En C/C++ las matrices se almacenan **por filas** en memoria. El loop i,j (*j* varía rápido) accede elementos contiguos :

Matriz A (4×8) de `double` — loop i,j

fila 0	[0][0]	[0][1]	[0][2]	[0][3]	[0][4]	[0][5]	[0][6]	[0][7]
fila 1	[1][0]	[1][1]	[1][2]	[1][3]	[1][4]	[1][5]	[1][6]	[1][7]
fila 2	[2][0]	[2][1]	[2][2]	[2][3]	[2][4]	[2][5]	[2][6]	[2][7]
fila 3	[3][0]	[3][1]	[3][2]	[3][3]	[3][4]	[3][5]	[3][6]	[3][7]

Memoria



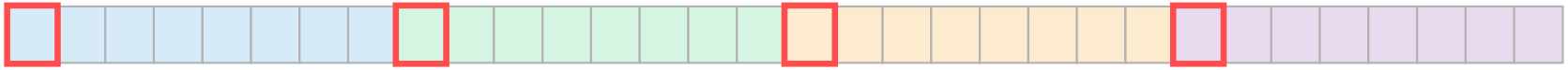
Loop j,i: saltos en memoria

Cada acceso del loop j,i (*i* varía rápido) carga una **cacheline distinta** (una por fila):

Matriz *A* (4×8) de `double` — loop j,i

cacheline 0	[0][0]	[0][1]	[0][2]	[0][3]	[0][4]	[0][5]	[0][6]	[0][7]
cacheline 1	[1][0]	[1][1]	[1][2]	[1][3]	[1][4]	[1][5]	[1][6]	[1][7]
cacheline 2	[2][0]	[2][1]	[2][2]	[2][3]	[2][4]	[2][5]	[2][6]	[2][7]
cacheline 3	[3][0]	[3][1]	[3][2]	[3][3]	[3][4]	[3][5]	[3][6]	[3][7]

Memoria



- Si la matriz cabe en caché: las cachelines se reutilizan en la siguiente columna.
- Si la matriz no cabe en caché: las cachelines se desalojan antes de recorrer la siguiente columna. Hay que recargarlas.

Midiendo con `perf stat`

Linux provee `perf`, una herramienta para medir *hardware performance counters*:

```
perf stat -e cache-misses,cache-references ./row_vs_col
```

Algunos contadores útiles:

- `cache-references`, `cache-misses`: accesos y fallos de caché.
- `L1-dcache-load-misses`: misses en caché L1 de datos.
- `LLC-load-misses`: misses en *Last Level Cache* (L3).
- `instructions`, `cycles`: instrucciones ejecutadas y ciclos de CPU.
- `branches`, `branch-misses`: predicciones de salto fallidas.

Listar todos los disponibles: `perf list`

`perf stat`: eventos

- `task-clock`: tiempo de CPU (ms) consumido por el proceso.
- `cycles`: ciclos de reloj del procesador.
- `instructions`: instrucciones ejecutadas.
  $\mathrm{IPC} = \texttt{instructions}/\texttt{cycles}$ indica eficiencia del procesador.
- `cache-references`: accesos al *last-level cache* (L3).
- `cache-misses`: accesos a L3 que no encontraron el dato. La razón con `cache-references` da la tasa de fallo.
- `L1-dcache-load-misses`: lecturas que fallan en L1.
- `LLC-load-misses`: lecturas que fallan en L3 y van a RAM ($\sim$60 ns).

Mapeo de caché: problema

Si un dato se guarda en el caché, ¿dónde se ubica?

Dirección RAM $\longrightarrow$ Dirección Caché

- ¿Cómo se determina en qué lugar de la memoria caché almacenaremos un determinado dato proveniente de la memoria RAM?
- No es posible asignar a cada dato un lugar distinto, porque RAM es mayor que Caché.
- Necesitamos un **mapeo** que, según de dónde viene el dato, asigne un lugar en la memoria caché.
- Este mapeo forma parte del **hardware**. ¿Qué puede significar esto?

Direcciones de memoria

Cada byte en memoria tiene una **dirección** única, representada en binario.

¿Cuántos bits necesitamos?

- Con n bits podemos distinguir 2^n direcciones distintas.
- RAM de 4 GiB $= 2^{32}$ bytes. Necesitamos **32 bits**.
- Caché de 64 KiB $= 2^{16}$ bytes. Necesitamos **16 bits**.

Una dirección de memoria es simplemente el número identificador de un byte.

Mapeo directo

Si un dato se guarda en el caché, ¿dónde se ubica?

Dirección RAM (32 bits) $\longrightarrow$ Dirección Caché (16 bits)

- **Mapeo directo:** tomamos los últimos 16 bits de la dirección.
- Es rápido y sencillo (solo un *and* de bits).
- Pero distintas direcciones de RAM que coincidan en sus últimos 16 bits se mapean al mismo lugar en caché.
- **Principio del palomar:** 2^{32} direcciones de RAM deben caber en 2^{16} posiciones de caché. Inevitablemente, múltiples direcciones comparten el mismo slot.

Mapeo directo: ejemplo

Supongamos un caché L1 de 64 KiB, con direcciones de 16 bits.

```
1     double A[3][8192];
2     for (i=0; i<8192; i++)
3       a[2][i] = ( a[0][i]+a[1][i] )/2.;
```

- Cada fila ocupa $8192 \times 8 = 64$ KiB: exactamente el tamaño del caché.
- Las direcciones de `a[0][i]`, `a[1][i]` y `a[2][i]` están separadas por exactamente 64 KiB $= 2^{16}$ bytes.
- Por lo tanto, sus últimos 16 bits coinciden: el mapeo directo las lleva al mismo slot.

Mapeo directo: ¿qué ocurre en el ciclo?

```
a[2][i] = ( a[0][i] + a[1][i] ) / 2.0
```

En cada iteración, los tres acceden al mismo mismo slot del caché:

1. Cargar `a[0][i]` de RAM a caché: *miss*. Llevar a registro r_0.
2. Cargar `a[1][i]` de RAM a caché (mismo slot): *miss*, **desaloja** `a[0][i]`. Llevar a registro r_1.
3. Sumar $r_0 + r_1$ y dividir por 2 en registros. Resultado en r_2.
4. Escribir r_2 en `a[2][i]`: cargar en caché (mismo slot), *miss*, **desaloja** `a[1][i]`.
5. Siguiente iteración: `a[0][i+1]` fue traído junto con `a[0][i]` en el paso 1 (misma línea de caché), pero fue desalojado en el paso 2.

   Resultado: 3 *misses* por iteración.

Cache miss por conflicto

Este es un ejemplo de **conflict miss**: direcciones distintas que se mapean al mismo slot de caché.

- Al acceder alternadamente a datos que comparten slot, cada acceso desaloja al anterior.
- La caché puede tener espacio libre, pero el mapeo directo no permite usarlo.
- **Solución**: caché *set-associative* o reorganizar los datos.

Completamente asociativo

Si un dato se guarda en el caché, ¿dónde se ubica?

Dirección RAM $\longrightarrow$ Dirección Caché

- Caché asociativo: los datos pueden ir donde sea (en la memoria caché)
- A esto se le llama **completamente asociativo**.
- Lamentablemente, es costoso de construir.

K-asociativo

Si un dato se guarda en el caché, ¿dónde se ubica?

Dirección RAM $\longrightarrow$ Dirección Caché

- Caché asociativo: los datos pueden ir a más de un lugar (en la memoria caché)
- A esto se le llama **K-asociativo**.
- $K \geq 2$.
- En Linux: `getconf -a | grep CACHE` y `lscpu | grep cache`

K-asociativo: ejemplo

¿Qué ocurre con el mismo código si ahora tenemos un caché **2-asociativo**?

```
1      double A[3][8192];
2      for (i=0; i<8192; i++)
3        a[2][i] = ( a[0][i]+a[1][i] )/2.;
```

- El caché sigue siendo 64 KiB, pero ahora tiene 4096 sets de 2 vías cada uno.
- Las filas siguen separadas por 64 KiB, pero el periodo de aliasing es ahora 2^{15} bytes (la mitad): `a[0][i]` y `a[1][i]` mapean al mismo set.
- Con 2 vías, `a[0][i]` y `a[1][i]` caben simultáneamente en el set.
- Pero `a[2][i]` es un tercero en el mismo set: desaloja a uno de los dos anteriores.
- Resultado: aún hay *conflict misses*, pero menos que con mapeo directo.

Políticas de reemplazo

Cuando la caché está llena y llega un dato nuevo, ¿qué línea se desaloja?

- **LRU** (*Least Recently Used*): se desaloja la línea que lleva más tiempo sin ser accedida.
  Buena aproximación, pero costosa de implementar en hardware para alta asociatividad.

- **FIFO** (*First In, First Out*): se desaloja la línea más antigua, sin importar cuántas veces fue accedida.

- **Random**: se elige una línea al azar. Sorprendentemente competitivo y muy barato de implementar.

En la práctica, los procesadores modernos usan variantes de **pseudo-LRU** (menos costoso que LRU puro).

Políticas de escritura

Cuando el procesador modifica un dato en caché, ¿cuándo se actualiza la memoria principal?

- ***Write-through***: cada escritura en caché se propaga inmediatamente a memoria.
  Coherencia simple, pero alto tráfico al bus de memoria.

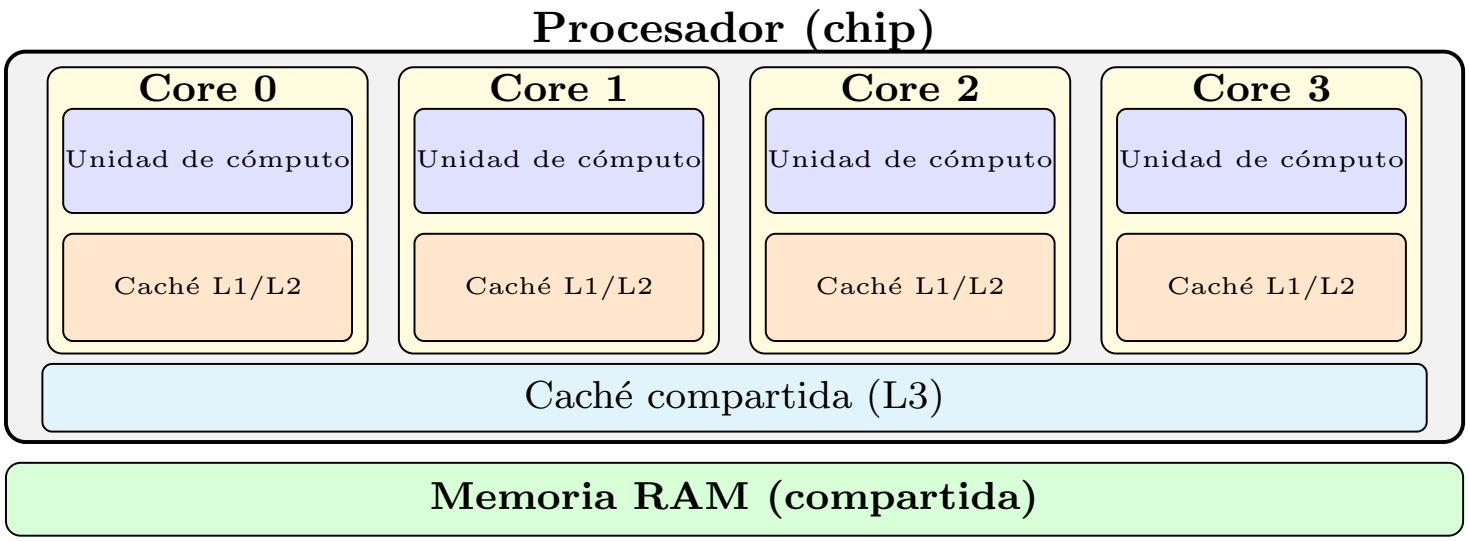
- ***Write-back***: la escritura se hace solo en caché; la línea se marca como *dirty* y se escribe a memoria solo al ser desalojada.
  Menos tráfico, pero coherencia más compleja en multicore.

La mayoría de los procesadores modernos usan ***write-back*** por eficiencia.

Estados de una línea de caché

- **Scratch**. La línea no contiene ningún dato válido.
- **Valid**. La línea es una copia correcta de memoria principal.
- **Dirty**. La línea ha sido modificada, pero aún no re-escrita en memoria.

Caché y multicore



Jerarquía de caché en multicore

- **L1 y L2** son privadas para cada núcleo: cada core tiene su propia copia, inaccesible directamente por los demás.
- **L3** es compartida entre todos los núcleos del procesador: todos leen y escriben en el mismo banco de caché.
- Datos en L1/L2 son más rápidos pero no visibles entre núcleos.

  Datos en L3 son visibles por todos, pero con mayor latencia.

Caché en sistemas multicore

En un procesador multicore, cada núcleo tiene su propia caché L1 (y a veces L2).

- Cada núcleo puede leer y escribir datos de forma independiente.
- La memoria RAM es **compartida** entre todos los núcleos.
- Si dos núcleos tienen copias del mismo dato en sus cachés locales y uno lo modifica, el otro tiene una copia desactualizada.
- Esto introduce un nuevo problema: ¿cómo mantener la **consistencia** entre las cachés?

Coherencia

- Existe el potencial de que ocurra un conflicto si más de un procesador tiene una copia de un mismo dato.
- **Coherencia caché**.
  Asegurarse que todos los datos en caché sean copias precisas de la memoria principal.
- Si un procesador modifica, el otro procesador debe actualizar su copia.
- Pueden existir niveles de caché **privados** para un procesador.

Estados respecto a un dato

Modelo MSI

Un modelo más simple es MSI:

- **Modified**. Ha sido modificado y necesita ser escrito.
- **Shared**. Presente en al menos un caché, no modificado.
- **Invalid**. No presente, o bien ha sido modificado en otro caché.

Operaciones en protocolos de coherencia

Las transiciones entre estados se producen por dos tipos de eventos:

Acciones del procesador local (no visibles en el bus):

- **PrRd**: el procesador lee un dato de su caché.
- **PrWr**: el procesador escribe un dato en su caché.

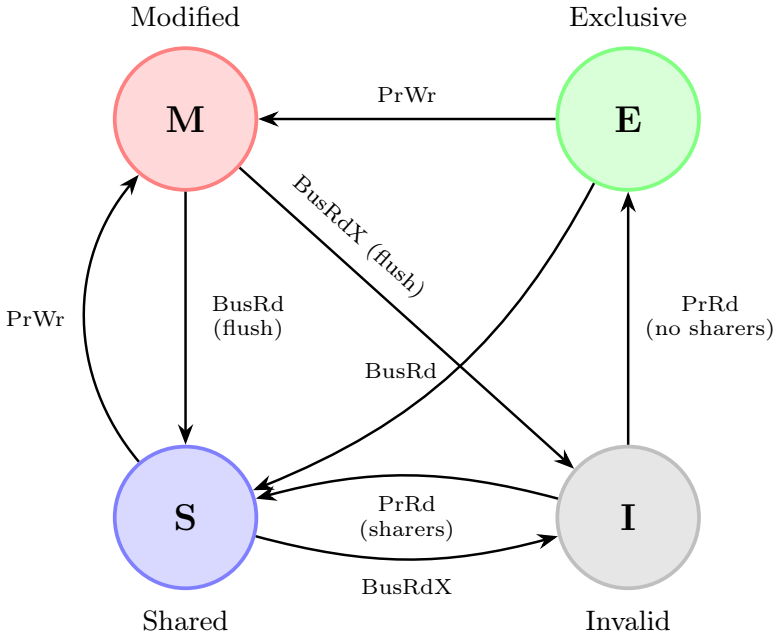
Transacciones en el bus (visibles para todos los núcleos):

- **BusRd**: un núcleo solicita leer un bloque; los demás observan la solicitud en el bus.
- **BusRdX**: un núcleo solicita leer un bloque *con intención de escribir*. Invalida todas las demás copias.

Cada núcleo *espía* (*snoops*) el bus: al detectar un BusRd o BusRdX, actualiza el estado de sus propias líneas de caché.

Modelo MESI

El protocolo **MESI** extiende MSI con el estado **Exclusive**:



- **Modified**: línea modificada localmente, no escrita a memoria.
- **Exclusive**: única copia, pero *limpia* (igual a memoria).
- **Shared**: presente en múltiples cachés, no modificada.
- **Invalid**: línea no presente o invalidada porque otro core modificó el dato.

El estado **Exclusive** permite escribir sin notificar al bus (pasa directo a Modified), reduciendo tráfico.